

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS143 - Estruturas de Dados II

02 - Análise de Algoritmos e Conectividade Dinâmica

Prof. Alex Kutzke

Agosto de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Contar operações de um trecho de código em função de n ;
2. Identificar o termo dominante de uma função de custo;
3. Classificar uma função com O , Ω e Θ ;
4. Distinguir melhor caso, pior caso e caso médio;
5. Estimar a ordem de crescimento por medição (hipótese de duplicação);
6. Explicar as três implementações de Union-Find e o custo de Find/Union em cada uma;
7. Justificar por que pendurar a árvore menor sob a maior reduz $O(n)$ para $O(\log n)$;
8. Escolher, para um tamanho de entrada dado, qual implementação usar.

Roteiro

Análise

Notação assintótica

Crescimento

Union-Find

Lado a lado

Três programas, a mesma resposta

Na Aula 1, o desafio foi implementar Find e Union. Existe mais de uma forma de fazer isso.

```
cd 02/codes/unionfind
go build -o quick_find quick_find.go
go build -o quick_union quick_union.go
go build -o weighted weighted_quick_union.go

./weighted < largeUF.txt > /dev/null
```

Agora o mesmo arquivo, com `quick_find`. Quanto tempo leva?

800 vezes mais lento

weighted: **cerca de 1 segundo**

quick_find: **quase 14 minutos**

Mesma entrada, mesma máquina, mesmo compilador, menos de 140 linhas cada. A diferença vem só da estrutura de dados escolhida.

Medir essa diferença é a primeira metade da aula. Explicar de onde ela vem é a segunda.

Como decidir sem esperar 14 minutos?

Análise empírica

Implementar, rodar, cronometrar. Indispensável, mas: depende dos dados de entrada, da máquina, do compilador. E custa tempo – não dá para implementar seis alternativas para escolher uma.

Como decidir sem esperar 14 minutos?

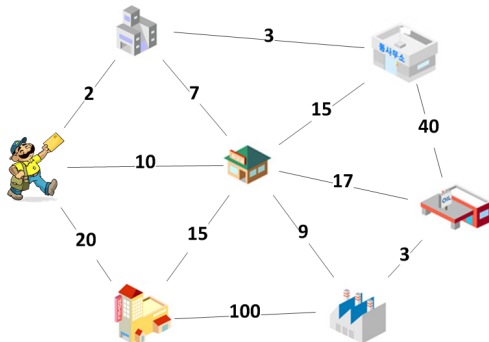
Análise empírica

Implementar, rodar, cronometrar. Indispensável, mas: depende dos dados de entrada, da máquina, do compilador. E custa tempo – não dá para implementar seis alternativas para escolher uma.

Análise de algoritmos

Estuda desempenho no papel, antes de existir código. Responde “quanto tempo para uma entrada de tamanho n ?” sem depender da máquina.

O problema do caixeiro viajante



Visitar n cidades, voltando à primeira. Qual rota minimiza a distância total?

Quantas rotas existem?

Com 4 cidades A, B, C, D (fixando A como origem): 6 rotas.

Quantas rotas existem?

Com 4 cidades A, B, C, D (fixando A como origem): 6 rotas.

E com 20 cidades?

$(n - 1)!$ rotas

n	$(n - 1)!$	tempo total (1 bilhão de add./s)
10	362.880	0,003 s
15	87 bilhões	20 min
20	$1,2 \times 10^{17}$	73 anos
25	$6,2 \times 10^{23}$	470 milhões de anos

$(n - 1)!$ rotas

n	$(n - 1)!$	tempo total (1 bilhão de add./s)
10	362.880	0,003 s
15	87 bilhões	20 min
20	$1,2 \times 10^{17}$	73 anos
25	$6,2 \times 10^{23}$	470 milhões de anos

Com uma máquina 1000 vezes mais rápida: 73 anos caem para 26 dias. Some **uma** cidade e o ganho desaparece.

Compare com um algoritmo n^5

n	n^5	tempo total
10	100.000	insignificante
15	759.375	0,01 s
20	3.200.000	0,06 s
25	9.765.625	0,23 s

Expoente alto, considerado lento – e ainda assim resolve em décimos de segundo o que o outro não resolve em milhões de anos.

Polinomial e fatorial são de naturezas diferentes. A distância entre n^2 e n^5 é pequena comparada a essa.

O que contar

Contar “instruções de máquina” não serve – depende do compilador e do processador.

Sedgewick propõe o **modelo de custo**: escolher a operação relevante para aquele algoritmo e contar só ela.

Ordenação	comparações e trocas
Busca	comparações com a chave
Union-Find	acessos ao array

Quantas vezes contador++ executa?

```
for i := 0; i < n; i++ {           // (a)
    contador++
}
for i := 0; i < n; i++ {           // (b)
    for j := 0; j < n; j++ { contador++ }
}
for i := 0; i < n; i++ {           // (c)
    for j := i; j < n; j++ { contador++ }
}
```

Responda antes de continuar.

n, n^2 e...

(a) n (b) n^2

n, n^2 e...

(a) n (b) n^2

(c) o laço interno começa em i : executa $n, n-1, \dots, 1$ vezes.

$$n + (n-1) + \dots + 1 = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

Metade do trabalho de (b) – ainda assim, os dois caem na mesma categoria.

Bubble Sort: quantas comparações no total?

```
func bubbleSort(v []int) int {  
    n := len(v)  
    comparacoes := 0  
    for i := 1; i < n; i++ {  
        for j := 0; j < n-i; j++ {  
            comparacoes++  
            if v[j] > v[j+1] {  
                v[j], v[j+1] = v[j+1], v[j]  
            }  
        }  
    }  
    return comparacoes  
}
```

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

$$\text{Comparações} = \sum_{i=1}^{n-1} (n - i)$$

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

$$\begin{aligned}\text{Comparações} &= \sum_{i=1}^{n-1} (n - i) \\ &= (n - 1) \cdot n - (1 + 2 + \cdots + (n - 1))\end{aligned}$$

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

$$\begin{aligned}\text{Comparações} &= \sum_{i=1}^{n-1} (n - i) \\ &= (n - 1) \cdot n - (1 + 2 + \cdots + (n - 1)) \\ &= (n^2 - n) - \frac{n(n - 1)}{2}\end{aligned}$$

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

$$\begin{aligned}\text{Comparações} &= \sum_{i=1}^{n-1} (n - i) \\ &= (n - 1) \cdot n - (1 + 2 + \cdots + (n - 1)) \\ &= (n^2 - n) - \frac{n(n - 1)}{2} \\ &= \frac{n^2}{2} - \frac{n}{2}\end{aligned}$$

Termo dominante

n	$n/2$	$n^2/2$	$n^2/2 - n/2$
1024	512	524.288	523.776
32768	16.384	536.870.912	536.854.528

Na última linha, $n/2$ é três milésimos de 1% do total: existe, e é irrelevante.

Duas simplificações

Descartar termos de ordem inferior: $n^2/2 - n/2 \rightarrow n^2/2$.

Descartar constantes multiplicativas: $n^2/2 \rightarrow n^2$.

Válido **quando** n é **grande**. Problema pequeno, qualquer algoritmo resolve.

O – limite superior

$f(n) = O(g(n))$ quando existem $c > 0$ e $n_0 \geq 0$ tais que

$$f(n) \leq c \cdot g(n) \quad \text{para todo } n \geq n_0$$

A partir de certo n , f nunca ultrapassa g vezes uma constante.

Bubble Sort: $n^2/2 - n/2 = O(n^2)$. Também é $O(n^3)$ – um limite frouxo continua sendo limite, mas informe sempre o mais justo que conhecer.

Ω e Θ

Ω – limite inferior

$f(n) = \Omega(g(n))$: a partir de certo n , f é **pelo menos** g vezes uma constante.

Θ – limite justo

$f(n) = \Theta(g(n))$ quando $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$.

Bubble Sort é $\Theta(n^2)$: o custo real, não só um limite.

No dia a dia, escreve-se O quase sempre – mesmo quando Θ seria a afirmação mais forte. Saiba a diferença, use O .

Classes de crescimento

Classe	Função	Exemplo	Ao dobrar n
Constante	1	acesso a $v[i]$	não muda
Logarítmica	$\log n$	busca binária	soma constante
Linear	n	percorrer um slice	dobra
Linearítmica	$n \log n$	MergeSort	pouco mais que dobra
Quadrática	n^2	Bubble Sort	quadruplica
Cúbica	n^3	três laços	multiplica por 8
Exponencial	2^n	subconjuntos	inviável
Fatorial	$n!$	caixeiro viajante	inviável antes

A base do log não aparece: mudar de base só multiplica por uma constante.

Melhor caso, pior caso, caso médio

Comparações do Bubble Sort não dependem do vetor. **Trocas** dependem:

- vetor já ordenado: 0 trocas (melhor caso);
- vetor decrescente: $n^2/2 - n/2$ trocas (pior caso).

QuickSort: $O(n^2)$ ou $O(n \log n)$?

Depende do caso. Pior caso: $O(n^2)$. Caso médio: $O(n \log n)$.
Omitir qual caso torna a afirmação inútil nas duas direções.

Como descobrir a classe sem ler o código?

Teste de duplicação

1. gere uma entrada aleatória de tamanho n ;
2. meça o tempo;
3. dobre n e repita;
4. observe para onde converge a razão entre tempos.

Meça o binário compilado, não `go run`. Descarte medições abaixo de um décimo de segundo.

Por que funciona

A maioria dos programas segue uma lei de potência: $T(n) = a \cdot n^b$.

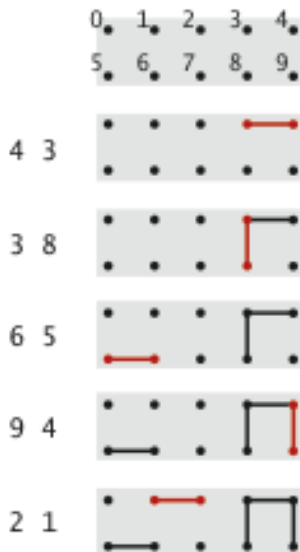
Dobrando a entrada:

$$\frac{T(2n)}{T(n)} = \frac{a \cdot (2n)^b}{a \cdot n^b} = 2^b$$

a desaparece na divisão. Só resta b .

Razão	2	4	8
Classe	linear	quadrática	cúbica

Retomando o problema



Quick-Find: a estrutura

`id[i]` guarda a componente de `i`. Conectados $\iff$ mesmo valor em `id`.

```
func (uf *UF) Find(p int) int {  
    return uf.id[p]           // O(1)  
}  
func (uf *UF) Connected(p, q int) bool {  
    return uf.id[p] == uf.id[q] // O(1)  
}
```

E Union, quanto custa?

Quick-Find: o preço do Union

```
func (uf *UF) Union(p, q int) {  
    pID, qID := uf.id[p], uf.id[q]  
    if pID == qID { return }  
    for i := 0; i < uf.n; i++ {    // percorre TUDO  
        if uf.id[i] == pID {  
            uf.id[i] = qID  
        }  
    }  
    uf.count--  
}
```

$O(n)$ por união, sempre – mesmo unindo dois elementos. M operações sobre n objetos: $O(M \cdot n)$, quadrático quando $M \propto n$.

Algoritmos quadráticos não acompanham a evolução do hardware (10x mais rápido resolve um problema 10x maior 10x mais lento).

Quick-Union: adiar o trabalho

Em vez de renomear tudo, muda-se **um** valor – o custo migra para Find.

```
func (uf *UF) Find(p int) int {
    for p != uf.parent[p] {
        p = uf.parent[p]
    }
    return p
}

func (uf *UF) Union(p, q int) {
    rootP, rootQ := uf.Find(p), uf.Find(q)
    if rootP == rootQ { return }
    uf.parent[rootP] = rootQ
    uf.count--
}
```

Union agora é uma atribuição. Qual o custo de Find?

Depende da altura da árvore

Union(0,1)

1

|

0

Union(1,2)

2

|

1

|

0

Union(2,3)

3

|

2

|

1

|

0

Union sempre pendura a raiz **antiga** sob a nova – a cadeia cresce por baixo. Find(0) percorre 3 arestas.

Pior caso: árvore degenera numa lista de n nós. Find custa n – **pior que Quick-Find**, e sem nem ser acesso contíguo à memória.

O defeito é identificável

Union escolhe **arbitrariamente** qual raiz pende de qual – não usa nenhuma informação sobre as árvores.

Como você corrigiria isso?

Pendurar a menor sob a maior

```
if uf.size[rootP] < uf.size[rootQ] {  
    uf.parent[rootP] = rootQ  
    uf.size[rootQ] += uf.size[rootP]  
} else {  
    uf.parent[rootQ] = rootP  
    uf.size[rootP] += uf.size[rootQ]  
}
```

Três linhas a mais que o Quick-Union.

Proposição

A profundidade de qualquer nó é no máximo $\log_2 n$.

A profundidade de x só aumenta quando a árvore de x é a **menor** das duas – e nesse momento o tamanho da árvore de x pelo menos dobra. Dobrar no máximo $\log_2 n$ vezes.

Custo de M operações sobre n objetos

Implementação	Custo	Ordem
Quick-Find	$M \cdot n$	quadrática
Quick-Union	$M \cdot n$	quadrática (pior caso)
Weighted Quick-Union	$n + M \log n$	linearítmica

Você mediria essas três com o teste de duplicação.
Que razão esperaria ver?

Medido de verdade

n	Quick-Find	razão	Weighted
40.000	1,23 s		0,02 s
80.000	4,96 s	4,0	0,06 s
160.000	19,63 s	4,0	0,12 s

Razão 4 confirma quadrático ($b = \log_2 4 = 2$), exatamente como a análise previu.

Em `largeUF.txt` (1 milhão de objetos): Weighted leva **1,07 s**. Quick-Find, **829 s** – os mesmos 14 minutos do início da aula, agora explicados.

Qual implementação usar

- Dezenas ou centenas de objetos: qualquer uma responde em microssegundos – escolha pela simplicidade do código;
- Milhares de objetos com uso intenso de `Union`: a ordem de crescimento domina, e `Weighted Quick-Union` é a escolha.

O ganho não vem de código mais rápido – as três têm o mesmo tipo de laço. Vem de escolher a estrutura de dados certa.

Resumo

- Medição depende de máquina, dados e implementação. Análise assintótica não depende de nenhum dos três;
- Analisar começa por declarar o modelo de custo;
- Termos de ordem inferior e constantes somem quando n cresce – para n pequeno, eles dominam;
- O é limite superior, Ω inferior, Θ os dois. Diga sempre qual caso;
- Razão ao dobrar n : 2 linear, 4 quadrática, 8 cúbica;
- Quick-Find: Find $O(1)$, Union $O(n)$. Quick-Union inverte, e degenera em árvore torta;
- Pendurar a menor sob a maior limita a profundidade a $\log_2 n$.

Bibliografia

- SEDGEWICK, R.; WAYNE, K. **Algorithms**, 4th ed., seções 1.4 e 1.5.
- <https://algs4.cs.princeton.edu/14analysis/>
- <https://algs4.cs.princeton.edu/15uf/>
- BHARGAVA, A. Y. **Entendendo Algoritmos**, cap. 1 e 4.